

Defensa por partes — TPO Programación I (Sanatorio San Roque)

1. Idea general del programa

Simulador de reserva de turnos por consola para un sanatorio ficticio. Por cada paciente: datos → especialidad y estudio → día y horario en una grilla → cobertura y costo → comprobante. Se repite en un menú con loop y al final se muestra un resumen de la sesión.

Por qué el programa está dividido en funciones y no en un solo bloque: cada paso del flujo es una responsabilidad distinta (pedir datos, elegir estudio, reservar turno, cobrar, imprimir). Con funciones cada parte se prueba y se explica sola, y el programa principal queda como una lista corta de llamadas que se lee de arriba abajo. La alternativa (todo suelto dentro del while) hubiera dado ~150 líneas en un solo bloque, imposible de repartir entre cuatro integrantes.

Por qué todo vive en memoria: el enunciado de esta entrega no habilita archivos. El estado se reparte en tres estructuras:

Estructura	Qué es	Para qué
DIAS_SEMANA, FRANJAS_HORARIAS	Listas simples (constantes)	Los rótulos de filas y columnas de la grilla
matriz_turnos	Matriz 5×6 de strings	Estado de la agenda: "Libre" / "Ocupado"
lista_turnos	Lista de listas	Historial de reservas de la corrida

2. Estructura del archivo

Constantes globales

DIAS_SEMANA, FRANJAS_HORARIAS

Funciones

```
datos_paciente()
seleccionar_estudio(especialidades, cardio, trauma, derma, pediatra)
mostrar_disponibilidad(matriz_turnos)
elegir_turno(matriz_turnos)
obra_social()
formatear_monto(monto)
generar_comprobante(datos, especialidad, estudio, dia, hora, obra_social_nombre, monto)
mostrar_resumen_turnos(lista_turnos)
```

Programa principal

```
inicialización (especialidades, estudios, matriz, lista_turnos)
loop del menú principal
```

Por qué las funciones van antes del programa principal: Python ejecuta el archivo de arriba abajo; una función tiene que estar definida antes de la línea que la llama. Poner el programa al final es la forma más simple de garantizarlo.

Por qué DIAS_SEMANA y FRANJAS_HORARIAS son globales y no parámetros: nunca cambian durante la ejecución (son constantes, por eso van en mayúsculas) y las usan tres funciones (mostrar_disponibilidad, elegir_turno, y el armado de la matriz en el programa principal). Pasarlas como parámetro en cada llamada agregaría ruido sin ganar nada, porque no hay riesgo de que otra parte del programa las modifique. En cambio matriz_turnos **sí** se pasa por parámetro porque es la que cambia (ver elegir_turno).

3. Detalle función por función

`datos_paciente()` MARTI

Qué función cumple: es la puerta de entrada de cada atención. Reúne los cuatro datos del paciente (nombre y apellido, edad, mail, DNI) y garantiza que lleguen válidos al resto del programa.

De qué manera:

1. Pide el dato con `input(...).strip()` (el `.strip()` saca espacios de más al principio y al final).
2. Mientras el dato sea inválido (`while`), muestra error y lo vuelve a pedir.
3. Cuando es válido, lo normaliza: nombre con `.title()` ("juan perez" → "Juan Perez"), mail con `.lower()`.
4. Los criterios: nombre de 5 a 25 caracteres; edad de 0 a 110; mail de 10 a 200 caracteres; DNI con `.isdigit()` y 7 u 8 dígitos.
5. Devuelve `[nombre_c, edad, dni, mail]`.

Por qué así y no de otra forma:

- **while de validación y no un if:** un `if` valida una sola vez; si el usuario se equivoca dos veces, el programa seguiría con un dato inválido. El `while` insiste hasta que el dato sirve. El patrón "pedir → validar → repetir" se repite cuatro veces a propósito: es simple y cualquiera del grupo lo puede explicar.
- **Condición con `or` en los rangos (`len < 5 or len > 25`):** el dato es inválido si falla *cualquiera* de los dos límites. Con `and` la condición nunca se cumple (nadie tiene un nombre a la vez menor a 5 y mayor a 25), y el error pasaría en silencio: es exactamente el bug que había en la primera versión y que se corrigió.
- **`.isdigit()` para el DNI en lugar de convertir a `int`:** convertir con `int()` rompería el programa si el usuario escribe letras, y como todavía no vimos `try/except`, no hay forma prolija de atrapar ese error. `.isdigit()` pregunta "¿son todos números?" sin romper. Además el DNI se guarda como string a propósito: no se hacen cuentas con él y así no se pierde ningún cero a la izquierda.
- **Devolver una lista y no cuatro valores sueltos:** una sola variable (`datos`) viaja por el programa y se la pasa entera a `generar_comprobante`. La alternativa de retornar cuatro valores obligaría a recibir cuatro variables en el principal y pasar cuatro parámetros más a cada función que las use. No se usó una tupla ni un diccionario porque todavía no los vimos.
- **`.title()` al guardar el nombre y no al imprimir:** se normaliza una vez, en el origen, y desde ahí todo el programa (`comprobante`, `resumen`) usa el nombre ya capitalizado. Si se formateara al imprimir habría que acordarse de hacerlo en cada `print`.
- **Limitación conocida (para no llevarse sorpresas):** la edad se lee con `int(input(...))`, que rompe si escriben texto. Se dejó así porque el manejo de errores (`try/except`) es tema posterior; con lo visto hasta ahora la única defensa sería validar con `.isdigit()` antes de convertir, y se priorizó no complicar el código con lo que no corresponde.

`seleccionar_estudio(especialidades, cardio, trauma, derma, pediatra)` MAURI

Qué función cumple: resuelve la pregunta "¿qué se va a hacer el paciente?". Es un menú de dos niveles: primero especialidad, después la práctica dentro de esa especialidad.

De qué manera:

1. Arma las opciones numeradas con una comprensión de listas:
`[f"{i + 1}. {especialidades[i]}" for i in range(len(especialidades))]` y las imprime.

2. Pide un número del 1 al 4 y lo valida con `while`.
3. Con `match` `eleccion` asigna a `lista_estudios` la lista de la especialidad elegida (`cardio`, `trauma`, `pediatria` o `derma`).
4. Repite el mismo armado de menú para los estudios de esa lista y pide otro número, validado contra `len(lista_estudios)`.
5. Devuelve `especialidades[eleccion - 1]`, `lista_estudios[estudio_opc - 1]`: dos strings ya resueltos (el texto, no el número).

Por qué así y no de otra forma:

- **match-case y no if/elif:** hace lo mismo, pero deja a la vista que es una elección entre casos discretos (1, 2, 3, 4) y no una cadena de condiciones. Es además una herramienta nueva de la materia, y la corrección pedía aprovecharla en vez de resolver todo con lo básico.
- **Comprensión de listas y no un for con append:** armar una lista transformando otra es exactamente el caso de uso. Son una línea en vez de cuatro (crear lista vacía, `for`, `append`, uso) y quedan menos lugares donde equivocarse.
- **Las listas de estudios llegan por parámetro:** dentro de la función no hay ningún dato escrito a mano. Si se agrega una especialidad nueva se cambia el programa principal y esta función solo necesita una rama más en el `match`. Si estuvieran adentro, la función mezclaría lógica (menú) con datos (catálogo).
- **eleccion - 1 para pasar de número a posición:** al usuario se le muestra una numeración desde 1 (natural para una persona) pero las listas empiezan en 0. La resta es el único punto de conversión y está siempre en la misma forma.
- **Devuelve texto y no números de opción:** el resto del programa (comprobante, resumen) necesita "Cardiología", no "1". Resolverlo acá evita que otras funciones tengan que conocer el orden de las especialidades.
- **Sacamos la línea que pisaba el parámetro:** en la primera versión la función recibía `especialidades` pero lo redefinía adentro con una lista nueva (el parámetro no servía de nada). Se eliminó y la lista se define una sola vez en el principal, sin el "1." pegado; la numeración se agrega al mostrar, no al guardar.
- **Por qué match y no un índice directo a una lista de listas:** se podría haber usado `[cardio, trauma, pediatria, derma][eleccion - 1]` y ahorrar el `match`. Se descartó porque es menos legible para quien lee el código por primera vez y porque depende de que el orden de las listas coincida en silencio con el de especialidades; el `match` deja esa correspondencia escrita a la vista.

`mostrar_disponibilidad(matriz_turnos)` JUANI

Qué función cumple: dibuja la agenda. Muestra todos los turnos de la semana de un vistazo, con "Libre" u "Ocupado" en cada casillero, para que el paciente elija sabiendo qué hay.

De qué manera:

1. Arma el encabezado: un espacio en blanco de 12 caracteres y luego cada franja horaria alineada a 10 (`f"{franja:<10}"`).
2. Recorre las filas de la matriz con `for d in range(len(DIAS_SEMANA))`. Cada fila arranca con el nombre del día (ancho 12) y le suma, franja por franja, el contenido de `matriz_turnos[d][h]`.
3. Imprime cada fila armada.

Por qué así y no de otra forma:

- **Función aparte, solo de lectura:** no modifica nada, solo imprime. Se separó de `elegir_turno` (que sí modifica) para que cada función haga una sola cosa. Beneficio concreto: se puede volver a llamar cuando haga falta sin efectos secundarios.

- **Ancho fijo con f"{valor:<N}" y no concatenar espacios a mano:** alinear a mano con espacios se desarma apenas cambia el largo de una palabra ("Libre" tiene 5 letras, "Ocupado" 7). El formato con ancho fijo rellena solo, y las columnas quedan derechas siempre.
 - **Recorrer con índices (range(len(...))) y no con for fila in matriz:** se necesita el índice d para conocer a la vez el nombre del día (DIAS_SEMANA[d]) y la fila de la matriz (matriz_turnos[d]). Con un for directo sobre la matriz habría que llevar un contador aparte.
 - **Armar el string de la fila y recién después imprimir:** un print por celda dejaría cada casillero en una línea distinta (a menos que se use end="", más difícil de leer). Acumular la fila en una variable y hacer un solo print es más limpio.
-

elegir_turno(matriz_turnos) JUANI

Qué función cumple: es el corazón del sistema de reservas. Convierte una elección del paciente (día y hora) en una **reserva real**: deja el casillero marcado como "Ocupado" para que nadie más lo tome.

De qué manera:

1. Muestra la grilla (llamando a mostrar_disponibilidad) y la lista de días numerados.
2. Entra en un while not turno_reservado: pide el día (1 a 5) validado con otro while.
3. Convierte la opción en índice: dia_idx = dia_opc - 1.
4. Calcula horas_libres con una comprensión de listas filtrada: las franjas de ese día cuya celda vale "Libre".
5. Si horas_libres está vacía, avisa y hace continue: vuelve a pedir un día sin salir del ciclo.
6. Si hay horarios, los muestra y pide uno; el while hora_elegida not in horas_libres rechaza tanto horarios inexistentes como ocupados.
7. Busca la posición con FRANJAS_HORARIAS.index(hora_elegida) y escribe matriz_turnos[dia_idx][franja_idx] = "Ocupado".
8. Pone turno_reservado = True (corta el ciclo) y devuelve DIAS_SEMANA[dia_idx], FRANJAS_HORARIAS[franja_idx].

Por qué así y no de otra forma:

- **La matriz se modifica dentro de la función y no se devuelve:** una lista en Python se pasa por referencia; matriz_turnos es el mismo objeto que vive en el programa principal, así que el cambio queda a la vista de todos sin retornarla. Si el segundo paciente intenta el mismo turno, ya no lo ve en horas_libres. Esto es lo que hace que la matriz sea un sistema de reservas y no un adorno para cumplir el requisito.
- **Validar contra horas_libres y no contra FRANJAS_HORARIAS:** validar contra todas las franjas dejaría escribir un horario que existe pero está ocupado, y habría que hacer un segundo chequeo aparte. Al filtrar primero, una sola condición (not in horas_libres) cubre "no existe" y "está ocupado" a la vez.
- **Bandera turno_reservado + continue y no un break:** el while externo se repite mientras no haya reserva; el caso "día sin horarios" necesita *reiniciar* la elección de día (continue), no salir. Con la bandera queda explícito en la condición del ciclo cuándo termina el proceso.
- **.index() para recuperar la columna:** el usuario tipea un texto ("10:00") pero la matriz se accede por posición. .index() traduce texto a posición sin recorrer a mano la lista. Se garantiza que existe porque el valor ya fue validado contra horas_libres, que sale de esa misma lista.
- **Comprensión de listas con if para filtrar:** es la versión corta y legible de "recorrer, preguntar, agregar". Se eligió sobre filter() porque es lo que se explicó en clase y es más fácil de leer de corrido.
- **Devolver día y hora como texto:** igual que en seleccionar_estudio, el comprobante necesita "Martes" y "10:00", no índices. Así el resto del programa no depende de cómo está armada la matriz.

- **Cuatro estados posibles cubiertos (para la defensa):** día inválido, día lleno, horario inválido, horario ocupado. Ninguno rompe el programa ni permite pisar una reserva.
-

obra_social() MARTI

Qué función cumple: define cuánto paga el paciente. Pregunta si tiene cobertura y devuelve el nombre de la entidad y el monto correspondiente.

De qué manera:

1. Pide "si" o "no" y lo normaliza con `.lower().strip()`; repite mientras no sea ninguna de las dos.
2. match respuesta:
 - "si": pide el nombre de la entidad (`.upper()`), y el monto es el coseguro: 3500.
 - "no": cobertura "Particular (Sin Obra Social)", monto 15000.
3. Devuelve nombre_os, monto_final.

Por qué así y no de otra forma:

- **match-case sobre "si"/"no":** son exactamente dos casos discretos de texto; queda paralelo al menú de seleccionar_estudio y usa lo nuevo de la materia.
 - **Precios como variables locales y no constantes globales:** solo se usan acá adentro. Exponerlos globalmente sugeriría que otras partes del programa los pueden usar o modificar, y no es así. Si el día de mañana otra función necesitara los precios, ahí sí se subirían a constantes.
 - **Normalizar con .lower() antes de comparar:** "SI", "Si" y "si" son la misma respuesta para una persona. Normalizar evita rechazar entradas correctas por mayúsculas.
 - **Nombre de la entidad en mayúsculas (.upper()):** "osde", "Osde" y "OSDE" quedan iguales en el comprobante. No se valida contra una lista de entidades reales porque el sistema no tiene convenios cargados; sería inventar datos.
 - **Un solo monto por caso y no un cálculo por práctica:** el enunciado no exige tarifario por estudio. Un precio fijo (particular) contra un coseguro fijo mantiene la función chica; agregar precios por práctica implicaría cambiar la firma y sumaría una estructura (tabla de precios) que no aporta al objetivo del TPO.
 - **Devuelve el monto como número y no como texto:** así se puede formatear después (`formatear_monto`) y, si hiciera falta, sumarlo (por ejemplo, totalizar la recaudación).
-

formatear_monto(monto) MAURI

Qué función cumple: convierte un número en texto con formato de moneda argentina: 15000 → "\$15.000".

De qué manera: `f"${monto:,.0f}"` escribe el número con coma como separador de miles y sin decimales (\$15,000); después `.replace(",", ".")` cambia la coma por el punto.

Por qué así y no de otra forma:

- **Función aparte, sin input ni print:** recibe un valor y devuelve un valor. Se usa desde `generar_comprobante` y desde `mostrar_resumen_turnos`; si el formato cambia (por ejemplo, agregar "ARS"), se edita en un solo lugar.
- **Formato de Python + .replace y no armar el separador a mano:** hacerlo a mano implicaría convertir a texto, recorrer desde el final y meter un punto cada tres dígitos: más código y más posibilidades de error. Python ya sabe agrupar miles; solo hay que traducir el símbolo a la convención local.

- **.0f (sin decimales):** los montos son enteros de pesos. Mostrar \$15.000,00 agregaría ruido sin información.
 - **Alternativa descartada, locale:** permitiría formato argentino directo, pero depende de que el sistema tenga instalada esa configuración regional, y no es lo que se vio en la materia. El .replace funciona igual en cualquier computadora.
-

generar_comprobante(datos, especialidad, estudio, dia, hora, obra_social_nombre, monto)
MARTI

Qué función cumple: produce el resultado visible de la reserva: el comprobante del turno con formato de ticket.

De qué manera:

1. Desempaqueta la lista datos en nombre_c, edad, dni, mail.
2. Arma una lista lineas, agregando cada renglón: separadores "=" * 45 y "-" * 45, título, y una línea por dato con la etiqueta alineada (f"{'Paciente:':<16}{nombre_c}").
3. El total lo formatea con formatear_monto(monto).
4. "\n".join(lineas) une todo en un único string con saltos de línea.
5. Lo imprime y lo devuelve.

Por qué así y no de otra forma:

- **Construir una lista de líneas y unirla al final, y no un print por renglón:** separa "armar el texto" de "mostrarlo". El comprobante queda como un valor (un string) que se puede reutilizar: mostrarlo, guardarlo más adelante en un archivo, mandarlo por mail. Con print sueltos ese texto solo existiría en pantalla.
 - **"=" * 45 para los separadores:** repetir un carácter con * evita escribir 45 signos a mano y garantiza que todos los separadores midan lo mismo; si se quiere un ticket más ancho se cambia un número.
 - **Etiquetas alineadas con :<16:** los valores empiezan todos en la misma columna, como un ticket real. Sin eso quedaría "Paciente: Juan" y "Especialidad: Cardiología" cada uno a distinta altura.
 - **Desempaquetar la lista en variables con nombre:** datos[0], datos[2] son ilegibles ("¿qué era la posición 2?"). Asignarlos a nombre_c, edad, dni, mail cuesta una línea y hace que el resto del código se lea solo.
 - **Además de imprimir, devuelve el string:** aunque hoy solo se imprima, devolverlo deja la función reutilizable y no cuesta nada.
 - **Por qué todos los datos llegan por parámetro:** la función no pide nada ni consulta nada; solo formatea lo que le pasan. Eso la hace fácil de probar (se puede llamar con datos inventados) y evita que dependa de variables del programa principal.
-

mostrar_resumen_turnos(lista_turnos) **LULI**

Qué función cumple: cierra la sesión mostrando el historial completo: todos los turnos reservados en una tabla, una fila por paciente.

De qué manera:

1. Imprime el encabezado con separadores.
2. Si lista_turnos está vacía (len == 0), avisa "No se registraron turnos".
3. Si no, recorre cada turno con un for, lo desempaqueta en siete variables (nombre_c, dni, especialidad, estudio, dia, hora, monto) y arma una fila con columnas de ancho fijo.
4. Imprime el separador final.

Por qué así y no de otra forma:

- **Lista de listas para el historial y no una segunda matriz:** `matriz_turnos` es una grilla de tamaño fijo (5×6) que representa el *estado* de la agenda. El historial, en cambio, crece con cada paciente y cada registro tiene datos de distinto tipo (textos y un número). Una lista a la que se le hace `append` en cada atención se adapta a eso sin tener que decidir de antemano cuántas filas habrá.
 - **Cada turno es una lista de 7 elementos:** el registro se arma en el principal con `lista_turnos.append([...])`. Se guardó solo lo que necesita la tabla (no el mail ni la edad, que no se muestran), para que el resumen sea liviano.
 - **Chequear la lista vacía primero:** si el usuario elige "Finalizar" sin atender a nadie, el `for` no imprimiría nada y la pantalla quedaría con dos separadores y nada en el medio. El mensaje evita esa confusión.
 - **Mismo patrón de ancho fijo que mostrar_disponibilidad:** se reutiliza una técnica ya usada en el programa en vez de inventar otra. Se conserva una única manera de alinear.
 - **Reutiliza formatear_monto:** el monto del resumen aparece igual que en el comprobante. Si se cambia el formato, se cambia en ambos a la vez.
 - **Función aparte del programa principal:** el resumen es una responsabilidad propia (leer el historial y mostrarlo) y no debería crecer dentro del bloque principal, que solo orquesta.
-

4. Programa principal — flujo completo JUANI

1. Saludo inicial
2. Se definen especialidades y las listas de estudios (cardio, trauma, derma, pediatria)
3. Se crea `matriz_turnos`: todas las celdas en "Libre" (comprensión de listas anidada)
4. Se crea `lista_turnos` vacía
5. Loop principal (while continuar):
 - a. Muestra menú: 1) Atender nuevo paciente 2) Finalizar y ver resumen
 - b. match-case sobre la opción:
 - 1: `datos_paciente` → `seleccionar_estudio` → `elegir_turno` → `obra_social` → `generar_comprobante` → se agrega el turno a `lista_turnos`
 - 2: `continuar = False`
6. Al salir del loop: `mostrar_resumen_turnos(lista_turnos)`

Por qué así y no de otra forma:

- **Matriz creada con comprensión de listas anidada** (`[["Libre" for _ in ...] for _ in ...]`) y no con `[["Libre"] * 6] * 5`: la multiplicación de listas copia la *referencia* de la fila, no la fila: las cinco filas serían la misma lista, y ocupar "Lunes 09:00" marcaría también Martes, Miércoles, Jueves y Viernes a las 09:00. La comprensión crea una fila nueva por cada día, independiente de las demás. Es el error clásico de las matrices y la razón concreta de por qué se armó así.
- **Los tamaños salen de len(FRANJAS_HORARIAS) y len(DIAS_SEMANA)**: si se agrega un día o una franja, la matriz se ajusta sola y no queda desalineada con los rótulos.
- **Los datos se crean antes del loop**: la matriz y `lista_turnos` tienen que sobrevivir de un paciente al siguiente. Si se crearan adentro del `while`, cada vuelta arrancaría con una agenda vacía y la reserva no bloquearía nada.
- **Bandera continuar y no while True con break**: la condición del ciclo dice a simple vista cuándo termina. Es más fácil de explicar y de seguir en una defensa oral.
- **match-case para el menú**: dos casos discretos; escalable si se suman opciones (por ejemplo "Cancelar turno"). Un `if/else` haría lo mismo pero se agranda peor.
- **El programa principal solo orquesta**: no tiene lógica de negocio; cada línea del caso 1 es una llamada a función. Eso permite leer el flujo completo del sistema en ocho líneas.

- **Se guarda una copia resumida en lista_turnos y no el comprobante entero:** el resumen final necesita siete campos, no el texto completo con sus adornos. Guardar solo lo necesario mantiene el historial simple de recorrer.
-